

Software, and Your Machine

Lab 1 — Operating systems, Unix, WSL, and getting set up

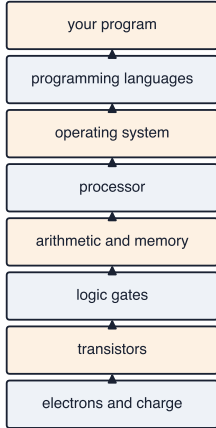
Amir Farbin

Part 1 — Software

Today

1. **Talk (~35 min)** — what an operating system actually is, why Unix, and why Windows users are about to install Linux
 2. **Do it (rest of the session)** — install the stack on *your* machine
 3. **Checkpoint** — Jupyter running, course materials cloned
- ▶ Everyone finishes the install today. Nobody leaves stuck.

Where we are



each layer is an
abstraction barrier:
solve it once,
build on top

The operating system

- ▶ Manages hardware; shares CPU, memory, devices among programs
- ▶ Programs ask the OS via an **API** — “make me a window”
- ▶ **Process** = running program + its resources
- ▶ **Preemptive multitasking**: the OS *takes* control back

Unix: born on mainframes

- ▶ 1970s Bell Labs, together with **C**
- ▶ Shared machines → multi-user + multitasking *from birth*
- ▶ 1980s open-source answers: **Linux** and **BSD**
- ▶ Unix is a **standard** — with a certification

The family tree

- ▶ **Distribution** = kernel + GNU tools + packaging (Ubuntu, Debian, RedHat)
- ▶ macOS = BSD kernel via **NeXT** (the Jobs detour) — certified Unix
- ▶ iOS = same system, tuned for a phone
- ▶ Windows: CP/M → DOS → Windows — single-user ancestry, retrofitted

The Unix philosophy

- ▶ Configuration is **plain text**
- ▶ **Everything is a file** — devices, programs, network streams
- ▶ Small tools (`ls`, `rm`, `mkdir`), composed in the **shell**
- ▶ Chain programs: the output of one is the input of the next

File systems (why this matters in 20 minutes)

- ▶ **Sectors** → **partitions** → **file systems**
- ▶ The **file** is an *invention*: named bits
- ▶ File table: name → blocks + metadata + **permissions**
- ▶ A whole file system can itself live inside **one file**

Two incompatibilities, not one

- ▶ Different **API**: a Linux program calls functions Windows doesn't have
- ▶ Different **instruction set**: ARM is not x86 machine code
- ▶ Which mismatch you have determines the fix

Four fixes

four ways to run one system's software on another

wrapper

translate API calls
one by one

WSL 1, Wine,
SteamOS

translation

translate machine
instructions

Rosetta 2,
Windows on ARM

virtual machine

simulate a whole
computer

WSL 2,
cloud servers

container

package just the
app + its needs

Docker, Colab

The ones you'll actually use

- ▶ **WSL 2**: a real Linux, in a VM, inside Windows — *two computers*
- ▶ The cloud = **hypervisors** booting your **image** on rented hardware
- ▶ **Containers**: package the app, share the kernel — Colab does this
- ▶ When software won't install: find the container

Why WSL, and why not Anaconda

- ▶ Anaconda *looks* easier on Windows, and it is not necessary
 - ▶ Everything we need installs with pip, in one command
 - ▶ It fakes a Unix-like environment, with limits you hit later
 - ▶ Hard for me and the TA to support
- ▶ **I want you on a real Unix as early as possible**
- ▶ On Windows, **WSL is required** for this course

Part 2 — Set It Up

What you are building

- ▶ A **Unix shell** you can type into
- ▶ **Python 3** and `pip`
- ▶ A **virtual environment** — the project's own private set of packages
- ▶ **JupyterLab** — notebooks in your browser
- ▶ A local copy of the **course repository**
- ▶ Windows: all of this lives inside WSL, not inside Windows

Windows: install WSL

In **PowerShell as Administrator** (right-click → Run as administrator):

```
wsl --install
```

- ▶ Installs WSL 2 and Ubuntu, then **reboot**
- ▶ Locked-down machine? Settings → Apps → Optional features → More Windows features → *Windows Subsystem for Linux*, then install **Ubuntu** from the Microsoft Store

Windows: first launch of Ubuntu

- ▶ Start menu → **Ubuntu**. First launch takes a minute
- ▶ It asks for a **username** and **password**
 - ▶ Lower-case username, no spaces
 - ▶ This is a *new, separate* account inside Linux
 - ▶ The password does not echo as you type — that is normal
- ▶ **Write both down.** You need the password for every `sudo`

macOS: the command line tools

Open **Terminal** (Applications → Utilities), then:

```
xcode-select --install
```

- ▶ Accept the dialog; it installs git, compilers, and friends
- ▶ macOS is certified Unix — you already have the shell
- ▶ Already installed? It says so. That's fine, move on

Linux: you're done

- ▶ You already have a shell and a package manager
- ▶ If you are not on Ubuntu or Debian, swap apt for your package manager
- ▶ Skip to the Python steps

Everyone: system packages

In the **Ubuntu terminal** (Windows) or **Terminal** (macOS/Linux).

Ubuntu / WSL / Debian:

```
sudo apt update
```

```
sudo apt install python3 python3-pip python3-venv git
```

macOS: Python 3 and git came with the command line tools.

▶ Then **close and reopen** the terminal

A change since last year: you can't pip install system-wide

```
sudo pip3 install numpy      # this now FAILS  
error: externally-managed-environment
```

- ▶ Modern Ubuntu **protects** the system Python from pip
- ▶ The system Python belongs to the OS; your packages are yours
- ▶ The fix is the right practice anyway: a **virtual environment**

A virtual environment

```
cd ~  
mkdir Data-3402  
cd Data-3402  
python3 -m venv .venv  
source .venv/bin/activate
```

- ▶ Your prompt now starts with (.venv)
- ▶ A private Python for this course. Nothing you install can break the system
- ▶ **Every new terminal:** cd ~/Data-3402 then source .venv/bin/activate

Install the Python stack

With `(.venv)` showing in your prompt:

```
pip install jupyterlab numpy matplotlib pandas
```

- ▶ No sudo. Inside the environment you are already allowed
- ▶ Takes a couple of minutes and prints a lot
- ▶ Check it:

```
python3 -c "import numpy, pandas, matplotlib; print('ok')"
```

Windows only: two disks, not one

- ▶ WSL's Linux lives in an **image file** on your Windows disk — it behaves as a whole separate volume
- ▶ Each side can see the other:
 - ▶ Windows → Linux: `\\wsl.localhost\Ubuntu` in File Explorer
 - ▶ Linux → Windows: `/mnt/c` is your C: drive
- ▶ Your **browser runs in Windows**, so downloads land on the Windows side
- ▶ **Do your work inside Linux** — your home directory, `~` — and *not* under `/mnt/c`; git and Jupyter are slow and buggy across the boundary

Get the course materials

```
cd ~/Data-3402
git clone https://github.com/UTA-DataScience/DATA3402.Fall.2026.git
cd DATA3402.Fall.2026
pip install -r requirements.txt
```

- ▶ That last line picks up the rest of the semester's packages
- ▶ This is a **read-only** copy: `git pull` brings you new lectures and labs
- ▶ You do **not** fork today, and you do **not** push here
- ▶ Submission goes through your own fork — we set that up together in the GitHub lab, and it will reuse this exact directory

Launch Jupyter

```
cd ~/Data-3402/DATA3402.Fall.2026  
jupyter lab
```

- ▶ On Windows it opens your Windows browser. Two computers, one browser
- ▶ If no browser opens, copy the `http://localhost:8888/...` link from the terminal and paste it in yourself
- ▶ Open `sample.ipynb`, click the code cell, press **Shift+Enter**
- ▶ **Ctrl-C** twice in the terminal to stop it

Checkpoint

Post one screenshot to the Teams channel showing:

1. JupyterLab open with `sample.ipynb` and Hello World! printed
2. A terminal where this prints ok:

```
python3 -c "import numpy, pandas, matplotlib; print('ok')"
```

► Stuck? Post the **exact error text**, not “it doesn’t work”

When it breaks

- ▶ `command not found: python3` → terminal not reopened, or the `apt install` step was skipped
- ▶ `externally-managed-environment` → not in the `venv` — activate it
- ▶ `ModuleNotFoundError: numpy` → same cause: no `(.venv)` in the prompt
- ▶ `sudo: incorrect password` → the **Linux** password, not your Windows one
- ▶ `wsl --install` refused → no admin rights: flag it, use Colab today
- ▶ Everything is slow → you're working under `/mnt/c`; move to your Linux home directory

If you're truly stuck today

- ▶ **Google Colab** works in a browser: notebooks, Python, the usual packages
- ▶ It is an acceptable fallback for the course, and it is *not* a substitute for having a working machine
- ▶ Come to office hours and we will fix the install properly

Before the next lecture

- ▶ Have the stack installed and Jupyter running
- ▶ Bring the machine you set up
- ▶ If your checkpoint screenshot isn't posted, post it as soon as it works
- ▶ Don't panic. Most of you had no trouble; the rest of you we fix together
- ▶ Next lab: a real tour of the Unix shell